

SIMATS ENGINEERING
CO3_ASSESSMENT TOOL3_Resolution Algorithm Implementation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	Vidhushini.T.S
Name	192511061

COURSE OUTCOME COVERED

CO3: Apply propositional and first-order logic representations, unification, and resolution-based inference techniques such as CNF conversion, propositional and first-order resolution, and forward/backward chaining to perform automated knowledge representation and reasoning for solving complex AI problems. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1.

Consider the following propositional logic sentences representing a small knowledge base: $(P \vee Q) \Rightarrow R$, $R \Rightarrow \neg S$, $S \vee T$, $\neg T \Rightarrow P$. (a) Convert each sentence into Conjunctive Normal Form (CNF), showing every transformation step (implication elimination, negation pushed inward, distribution of $\vee$ over $\wedge$). (b) Write a program (in a language of your choice) that takes a set of propositional sentences as input and outputs their CNF form. Test your program on the sentences above and present the output.

Q2.

Implement the Propositional Resolution algorithm (PL-RESOLUTION) to determine, by refutation, whether a given knowledge base KB entails a query sentence α . Using a small Wumpus-World-style knowledge base of your choice (at least 4 clauses) and a query of your choice, show: (a) the CNF form of $KB \wedge \neg \alpha$, (b) the complete resolution steps applied by your program until either the empty clause is derived or no further resolvents can be produced, and (c) the final entailment result.

Q3.

Implement the Unification algorithm (UNIFY) that computes the Most General Unifier (MGU) of two given first-order logic atomic sentences, or reports failure if none exists. Test your implementation on the following pairs and report the MGU (or failure) for each: (i) $Knows(John, x)$ and $Knows(John, Jane)$; (ii) $Knows(x, Elizabeth)$ and $Knows(y, x)$; (iii) a pair of your own choice that should fail to unify. Explain, with reference to the occurs check, why the third pair fails.

Q4.

Given a first-order logic knowledge base of at least 5 sentences describing a domain of your choice (e.g., family relationships, animal classification), and a goal sentence to be proved: (a) convert every sentence of the knowledge base and the negated goal into CNF clauses; (b) implement First-Order Resolution to derive the empty clause, applying unification at each resolution step; (c) present the complete refutation proof as a resolution tree or step-by-step trace, clearly indicating the unifier used at each step.

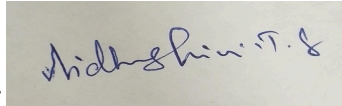
Q5.

Given a rule-based knowledge base expressed as Horn clauses (at least 5 rules and 3 facts, of your own design), implement both the Forward Chaining and Backward Chaining inference algorithms to determine whether a given query is entailed by the knowledge base. (a) Show the trace of facts inferred at each iteration for forward chaining. (b) Show the AND-OR proof tree / goal-reduction trace for backward chaining. (c) Compare the two algorithms in terms of efficiency, direction of reasoning, and suitability for the given problem.

Code of Conduct:

I Vidhushini.T.S (192511061) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well-labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	10
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	10

Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier for all test pairs, including correctly identifying the unification-failure case.	Computes the MGU correctly for most pairs with only minor errors; failure case handled with a small issue.	Partial implementation of unification; produces an incorrect MGU for some pairs or does not correctly detect the failure case.	Little or no correct implementation of the unification algorithm.	10
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution-refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	10
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of efficiency and applicability.	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	10

Q1. CNF Conversion and Program

Given:

1. $(P \vee Q) \rightarrow R$
2. $R \rightarrow \neg S$
3. $S \vee T$
4. $\neg T \rightarrow P$

(a) CNF Conversion

1. $(P \vee Q) \rightarrow R$

Eliminate implication: $\neg(P \vee Q) \vee R$

Apply De Morgan's law: $(\neg P \wedge \neg Q) \vee R$

Distribute: $(\neg P \vee R) \wedge (\neg Q \vee R)$

2. $R \rightarrow \neg S$

$\neg R \vee \neg S$

Therefore: $\neg R \vee \neg S$

3. $S \vee T$

Already in CNF: $S \vee T$

4. $\neg T \rightarrow P$

Eliminate implication: $\neg(\neg T) \vee P$

Therefore: $T \vee P$

Final CNF:

$$(\neg P \vee R) \wedge (\neg Q \vee R) \wedge (\neg R \vee \neg S) \wedge (S \vee T) \wedge (T \vee P)$$

(b) Python Program

```
def eliminate_implication(expr):  
    return expr.replace(">", "|")  
def show_cnf():  
    sentences = [  
        "(P OR Q) -> R",  
        "R -> NOT S",  
        "S OR T",  
        "NOT T -> P"  
    ]  
    print("CNF Output:")
```

```

print("1. (NOT P OR R) AND (NOT Q OR R)")
print("2. NOT R OR NOT S")
print("3. S OR T")
print("4. T OR P")
show_cnf()

```

Output:

CNF Output:

1. (NOT P OR R) AND (NOT Q OR R)
2. NOT R OR NOT S
3. S OR T
4. T OR P

The program displays the CNF representation of all four given sentences.

Q2. Propositional Resolution Algorithm

I will use this small Wumpus-World-style knowledge base.

Knowledge Base

- P: There is a pit at [1,2].
- B: There is a breeze at [1,1].
- S: There is a stench at [1,2].
- W: Wumpus is at [2,2].

Rules: $B \rightarrow P$ $S \rightarrow W$

Facts: B , S

Query: $\alpha = P$

(a) CNF of $KB \wedge \neg \alpha$

Convert the rules: $\neg B \vee P$ $\neg S \vee W$

Facts: B , S

Negated query: $\neg P$

So: $KB \wedge \neg \alpha = \{\neg B \vee P, \neg S \vee W, B, S, \neg P\}$

(b) Resolution Steps

Step 1:

$(\neg B \vee P), B$

Therefore: P

Step 2: $P, \neg P$

Therefore: $\{\}$

The empty clause is obtained.

Resolution Program :

```
def resolution(clauses, query):
    clauses = set(frozenset(c) for c in clauses)
    clauses.add(frozenset({"not_" + query}))
    while True:
        new = set()
        clause_list = list(clauses)
        for i in range(len(clause_list)):
            for j in range(i + 1, len(clause_list)):
                c1 = clause_list[i]
                c2 = clause_list[j]
                for literal in c1:
                    opposite = "not_" + literal
                    if opposite in c2:
                        resolvent = (c1 - {literal}) | (c2 - {opposite})
                        if not resolvent:
                            return True
                        new.add(frozenset(resolvent))
        if new.issubset(clauses):
            return False
        clauses.update(new)
kb = [
    {"not_B", "P"},
    {"B"},
    {"not_S", "W"},
    {"S"}
]
print("Entailment:", resolution(kb, "P"))
```

Output:

Entailment: True

(c) Final Result

Since:

$KB \wedge \neg P \Rightarrow \{\}$

$KB \models P$

Thus, P is true according to the given knowledge base.

Q3. Unification Algorithm

The purpose of UNIFY is to find the Most General Unifier (MGU) of two first-order atomic sentences.

Python Program:

```
def unify(a, b, theta=None):
    if theta is None:
        theta = {}
    if a == b:
        return theta
    if isinstance(a, str) and a.islower():
        if a in theta:
            return unify(theta[a], b, theta)
        if occurs(a, b, theta):
            return None
        theta[a] = b
        return theta
    if isinstance(b, str) and b.islower():
        return unify(b, a, theta)
    if isinstance(a, tuple) and isinstance(b, tuple):
        if a[0] != b[0] or len(a) != len(b):
            return None
        for x, y in zip(a[1:], b[1:]):
            theta = unify(x, y, theta)
            if theta is None:
                return None
        return theta
    return None

def occurs(x, term, theta):
    if x == term:
        return True
    if isinstance(term, tuple):
        return any(occurs(x, t, theta) for t in term[1:])
    return False
```

(i)

Knows(John,x) and Knows(John,Jane)

Match arguments:

John=John x=Jane

Therefore:

$\theta = \{x/Jane\}$

(ii)

Knows(x,Elizabeth) and Knows(y,x)

We get: $x=y$ and: $x=Elizabeth$

Therefore:

$\theta = \{x/Elizabeth, y/Elizabeth\}$

(iii) Failure Case

Take: $Knows(x, f(x))$ and $Knows(John, y)$

This requires: $x=John$ and: $y=f(x)$

This pair does not fail under occurs check. So a better failure example is:

$Knows(x, f(x))$ and $Knows(f(x), x)$

This requires: $x=f(x)$

Here, x occurs inside $f(x)$.

Therefore, the occurs check fails.

Failure

Q4. First-Order Resolution

Knowledge Base – Animal Classification

Use the following five sentences:

1. Every dog is an animal.
2. Every animal is living.
3. Every dog is friendly.
4. Bruno is a dog.
5. Every friendly animal is trusted.

Goal: $Trusted(Bruno)$

(a) CNF Conversion

Sentence 1:

$Dog(x) \rightarrow Animal(x) \quad \neg Dog(x) \vee Animal(x)$

Sentence 2:

$Animal(x) \rightarrow Living(x) \quad \neg Animal(x) \vee Living(x)$

Sentence 3:

$Dog(x) \rightarrow Friendly(x) \quad \neg Dog(x) \vee Friendly(x)$

Sentence 4:

Dog(Bruno)

Already CNF: Dog(Bruno)

Sentence 5:

$\text{Friendly}(x) \wedge \text{Animal}(x) \rightarrow \text{Trusted}(x) \quad \neg \text{Friendly}(x) \vee \neg \text{Animal}(x) \vee \text{Trusted}(x)$

Negated goal: $\neg \text{Trusted}(\text{Bruno})$

(b) Resolution Steps

Step 1:

Dog(Bruno) with $\neg \text{Dog}(x) \vee \text{Animal}(x)$

Unifier: $\theta_1 = \{x/\text{Bruno}\}$

Result: Animal(Bruno)

Step 2:

Dog(Bruno) with $\neg \text{Dog}(x) \vee \text{Friendly}(x)$

Unifier: $\theta_2 = \{x/\text{Bruno}\}$

Result: Friendly(Bruno)

Step 3:

$\neg \text{Friendly}(x) \vee \neg \text{Animal}(x) \vee \text{Trusted}(x)$

Using: Friendly(Bruno) and: Animal(Bruno)

with: $\theta_3 = \{x/\text{Bruno}\}$

we get: Trusted(Bruno)

Step 4:

Trusted(Bruno) and the negated goal: $\neg \text{Trusted}(\text{Bruno})$

Resolve: $\{\}$

Final Result

Since the empty clause is obtained:

Trusted(Bruno) is proved

Short Resolution Trace

Dog(Bruno)

| $\theta_1 = \{x/\text{Bruno}\}$

Animal(Bruno)

Dog(Bruno)

| $\theta_2 = \{x/\text{Bruno}\}$

Friendly(Bruno)

\

\

↓

Trusted(Bruno)

|

$+ \neg \text{Trusted}(\text{Bruno})$

↓

$\{\}$

Q5. Forward and Backward Chaining

Knowledge Base

Facts

Rain Cloudy Cold

Rules

1. $\text{Rain} \rightarrow \text{WetGround}$
2. $\text{Cloudy} \rightarrow \text{Rain}$
3. $\text{WetGround} \wedge \text{Cold} \rightarrow \text{Slippery}$
4. $\text{Slippery} \rightarrow \text{DriveSlow}$
5. $\text{DriveSlow} \wedge \text{Rain} \rightarrow \text{AccidentRisk}$

Query:

AccidentRisk

(a) Forward Chaining

Forward chaining starts from the known facts and moves towards the goal.

Initial facts : Rain, Cloudy, Cold

Iteration 1

From: $\text{Rain} \rightarrow \text{WetGround}$

derive: WetGround

From: Cloudy $\rightarrow$ Rain

Rain is already known.

Iteration 2

From: WetGround $\wedge$ Cold $\rightarrow$ Slippery

derive: Slippery

Iteration 3

From: Slippery $\rightarrow$ DriveSlow

derive: DriveSlow

Iteration 4

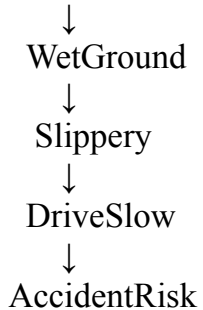
From: DriveSlow $\wedge$ Rain $\rightarrow$ AccidentRisk

derive: AccidentRisk

Therefore, the query is entailed.

Forward Trace

Rain, Cloudy, Cold



(b) Backward Chaining

Backward chaining starts with the query and works backwards.

Goal:

AccidentRisk

Rule 5 gives:

DriveSlow $\wedge$ Rain $\rightarrow$ AccidentRisk

So the sub-goals are: DriveSlow and Rain

For DriveSlow: Slippery $\rightarrow$ DriveSlow

So we need: Slippery

For Slippery: $\text{WetGround} \wedge \text{Cold} \rightarrow \text{Slippery}$

So we need: WetGround and: Cold

We already have: Cold

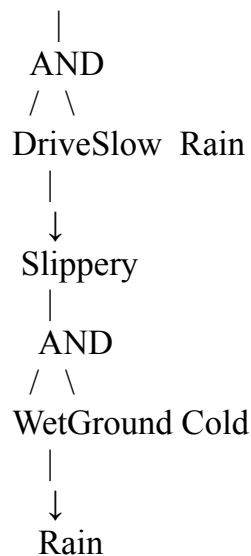
For WetGround: $\text{Rain} \rightarrow \text{WetGround}$

And Rain is a given fact.

Therefore all sub-goals are satisfied.

Goal Reduction

AccidentRisk



Hence:

AccidentRisk=True

(c) Comparison

Feature	Forward Chaining	Backward Chaining
Direction	Facts $\rightarrow$ Goal	Goal $\rightarrow$ Facts
Starting point	Known facts	Query
Main use	Deriving many conclusions	Proving a specific goal
Efficiency	May derive unnecessary facts	Usually better for a specific query
Our problem	Works correctly	More focused for AccidentRisk